

Vulnerability Triage Report

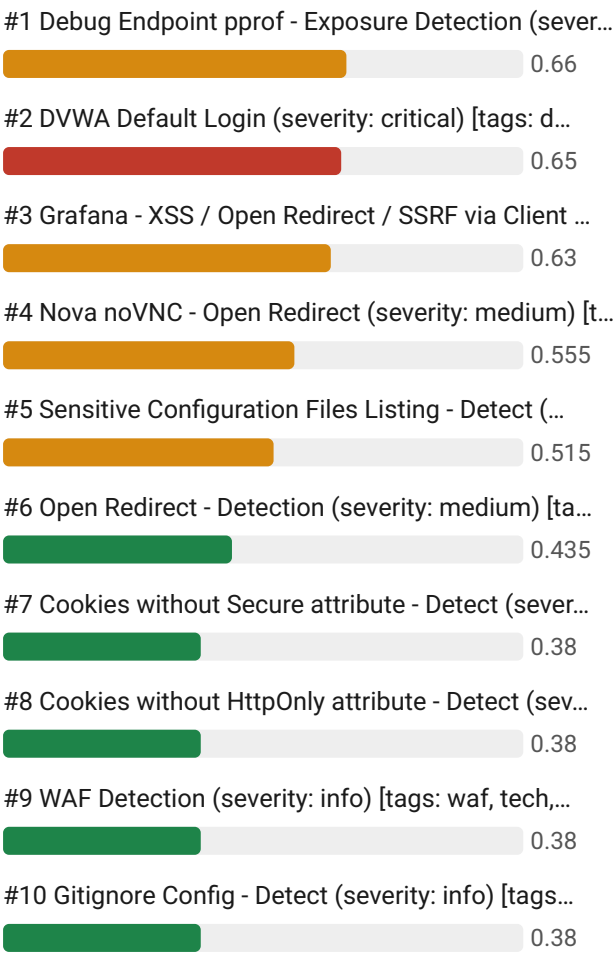
Generated by vulntrriage · 2026-07-09 19:13

Executive Summary

This report covers 47 vulnerability finding(s) triaged by the LLM-driven pipeline. Of these, 1 are rated High exploitability, 4 Medium, and 42 Low. The highest-priority finding is: Debug Endpoint pprof - Exposure Detection (severity: high) [tags: cve2019, cve, debug, kubernetes, kubelet, devops, unauth, disclosure, vkev, vuln] (host 172.19.0.3, risk score 0.66). Findings are ranked by a composite risk score combining CVSS, LLM-assessed exploitability, and asset criticality. Recommended remediation steps follow each finding.

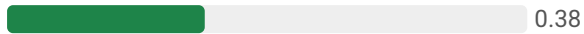


Risk Breakdown



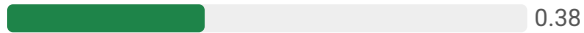
#11 FingerprintHub Technology Fingerprint (severity: info) [tags: fingerprint, technology]	0.38
#12 Missing Cookie SameSite Strict (severity: info) [tags: cookie, security]	0.38
#13 Wappalyzer Technology Detection (severity: info) [tags: technology, detection]	0.38
#14 README.md file disclosure (severity: info) [tags: file, disclosure]	0.38
#15 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#16 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#17 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#18 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#19 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#20 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#21 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#22 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#23 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#24 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38
#25 robots.txt endpoint prober (severity: info) [tags: robots.txt, endpoint, prober]	0.38
#26 robots.txt file (severity: info) [tags: robots.txt, file]	0.38
#27 Cookies without Secure attribute - Detect (severity: info) [tags: cookies, secure, attribute, detect]	0.38
#28 Cookies without HttpOnly attribute - Detect (severity: info) [tags: cookies, httponly, attribute, detect]	0.38
#29 SNMPv3 Fingerprint - Detect (severity: info) [tags: snmpv3, fingerprint, detect]	0.38
#30 HTTP Missing Security Headers (severity: info) [tags: http, security, headers]	0.38

#31 HTTP Missing Security Headers (severity: info) [...]



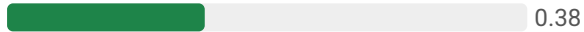
0.38

#32 HTTP Missing Security Headers (severity: info) [...]



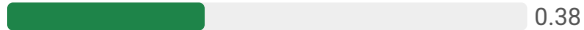
0.38

#33 HTTP Missing Security Headers (severity: info) [...]



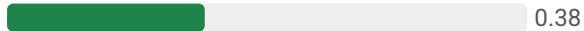
0.38

#34 HTTP Missing Security Headers (severity: info) [...]



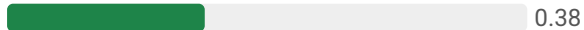
0.38

#35 HTTP Missing Security Headers (severity: info) [...]



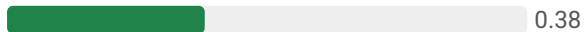
0.38

#36 HTTP Missing Security Headers (severity: info) [...]



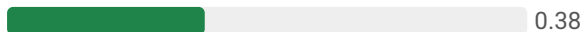
0.38

#37 HTTP Missing Security Headers (severity: info) [...]



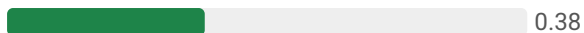
0.38

#38 HTTP Missing Security Headers (severity: info) [...]



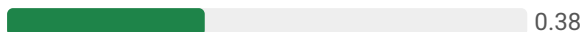
0.38

#39 FingerprintHub Technology Fingerprint (severity:...



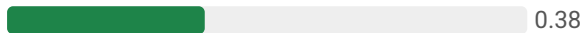
0.38

#40 Wappalyzer Technology Detection (severity: info)...



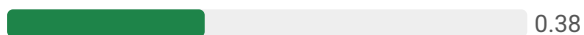
0.38

#41 Wappalyzer Technology Detection (severity: info)...



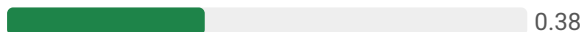
0.38

#42 Wordpress Plugin Detection (severity: info) [tag...



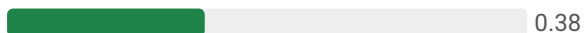
0.38

#43 Wordpress Plugin Detection (severity: info) [tag...



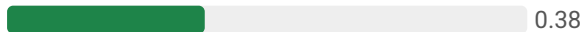
0.38

#44 Missing Cookie SameSite Strict (severity: info) ...



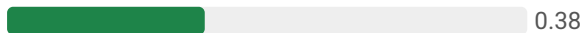
0.38

#45 FingerprintHub Technology Fingerprint (severity:...



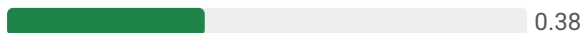
0.38

#46 Semantic UI Framework - Detect (severity: info) ...



0.38

#47 Form Detection (severity: info) [tags: form, mis...



0.38

Technical Findings

#1 Medium **Debug Endpoint pprof - Exposure Detection (severity: high)**
[tags: cve2019, cve, debug, kubernetes, kubelet, devops, unauth, disclosure, vkev, vuln]

Host: 172.19.0.3

CVE: cve-2019-11248

CVSS: 8.2

Risk score: 0.66 (asset criticality 0.5)

Context: The kubelet component in Kubernetes exposes an unauthenticated debug endpoint (pprof), allowing unauthorized access due to CVE-2019-11248. Attackers can exploit this to extract sensitive runtime data, such as environment variables or memory dumps, which may contain credentials or system configurations. This exposure can lead to data breaches, unauthorized cluster access, and potential disruption of services, posing significant risks to business operations and compliance.

Exploitability rationale: The unauthenticated debug endpoint has known exploits but is on a private network, requiring network access for exploitation.

Remediation: Disable or restrict access to the kubelet pprof endpoint to mitigate unauthenticated information disclosure.

1. Disable the pprof endpoint by setting `--profiling=false` in the kubelet configuration.
2. Enable authentication for the kubelet API using methods like X.509 certificates or tokens.
3. Implement authorization with RBAC or node authorizers to restrict access to authorized entities.
4. Apply network policies or firewall rules to limit access to the kubelet API port (e.g., 10250).
5. Update the Kubernetes cluster to the latest version with security patches applied.

#2 High DVWA Default Login (severity: critical) [tags: dvwa, default-login, vuln]

Host: 172.19.0.2

Risk score: 0.65 (asset criticality 0.5)

Context: The DVWA application is vulnerable due to default login credentials that are publicly known and easily guessable. Attackers can exploit this by using default credentials to gain unauthorized access, potentially leading to data theft, system compromise, or further malicious activities. Business impacts include risks of sensitive data breaches, reputational damage, and financial losses from regulatory penalties or operational disruptions.

Exploitability rationale: Default credentials are publicly known and easily guessable with very low attack complexity, making exploitation straightforward if the service is accessible, despite potential internal exposure.

Remediation: Remediate by immediately changing default credentials and hardening authentication to prevent unauthorized access.

1. Log into the DVWA application using default credentials (e.g., admin/password) to confirm the vulnerability exists.
2. Change all default passwords to strong, unique passwords that meet complexity requirements (e.g., at least 12 characters, including uppercase, lowercase, numbers, and symbols).
3. Review and update the DVWA configuration files or database settings to ensure no default credentials are stored or used.
4. Test the application by attempting to log in with old credentials (should fail) and new credentials (should succeed) to validate remediation.
5. Implement ongoing security measures such as regular password rotation, account lockout policies, and monitoring of authentication logs for suspicious activity.

#3 Medium Grafana - XSS / Open Redirect / SSRF via Client Path Traversal (severity: high) [tags: cve, cve2025, grafana, redirect, unauth, oss, vkev, vuln]

Host: 172.19.0.3

CVE: cve-2025-4123

CVSS: 7.6

Risk score: 0.63 (asset criticality 0.5)

Context: The Grafana platform is vulnerable to XSS, Open Redirect, and SSRF via client-side path traversal (CVE-2025-4123). Real-world attacks could involve injecting malicious scripts to steal user credentials, redirecting users to phishing sites, or accessing internal systems through SSRF. This poses high risks including data breaches, unauthorized access, and potential regulatory penalties.

Exploitability rationale: Internal service with known CVE and unauthenticated access, but requires network presence to exploit.

Remediation: Upgrade Grafana to a patched version to mitigate the client path traversal vulnerability that leads to XSS, open redirect, and SSRF.

1. Identify the current Grafana version on host 172.19.0.3 by checking service details or configuration files.
2. Consult Grafana Labs' official security advisory for CVE-2025-4123 to determine the affected versions and the recommended patched version.
3. Upgrade Grafana to the specified patched version or the latest stable release that includes the fix.
4. Restart the Grafana service and verify the upgrade by confirming the new version and testing for the vulnerability.
5. Implement additional hardening, such as input validation and disabling unused features, to reduce exposure until the upgrade is complete.

#4 Medium **Nova noVNC - Open Redirect (severity: medium) [tags: cve2021, cve, redirect, novnc, seclists, openstack, vuln]**

Host: 172.19.0.3

CVE: cve-2021-3654

CVSS: 6.1

Risk score: 0.555 (asset criticality 0.5)

Context: The vulnerability exists in Nova noVNC within OpenStack, identified as CVE-2021-3654, which allows open redirect attacks. In real-world scenarios, attackers could exploit this by crafting malicious URLs to redirect users to phishing sites or malware downloads, potentially leading to credential theft or data breaches. This poses a business risk of reputational damage, financial losses from remediation, and regulatory penalties. Organizations should apply patches and educate users to verify URLs before clicking.

Exploitability rationale: The internal network exposure and social engineering requirement moderate the exploitability despite the availability of public exploits for this CVE.

Remediation: Update OpenStack Nova and noVNC to patched versions and enforce input validation to prevent open redirects.

1. Check the current versions of OpenStack Nova and noVNC against affected versions listed in OpenStack security advisory for CVE-2021-3654.
2. Apply the latest security updates from OpenStack by upgrading Nova and noVNC to versions that address CVE-2021-3654.
3. If immediate update is not feasible, configure noVNC to validate and restrict redirect URLs to a whitelist of trusted domains.
4. Restart the affected services to apply changes and verify that the open redirect vulnerability is no longer exploitable.
5. Monitor security advisories and periodically review updates to maintain protection against similar vulnerabilities.

#5 Medium Sensitive Configuration Files Listing - Detect (severity: medium) [tags: config, listing, exposure, edb, vuln]

Host: 172.19.0.2

CVSS: 5.3

Risk score: 0.515 (asset criticality 0.5)

Context: The vulnerability exposes sensitive configuration files through directory listing or misconfigured services on the host. Attackers can exploit this to harvest credentials, API keys, or internal paths, enabling further attacks like unauthorized access or data theft. The business impact includes potential data breaches, financial losses, regulatory penalties, and damage to organizational reputation.

Exploitability rationale: Internal service exposure with low attack complexity and significant impact from misconfiguration, but requires access to the internal network.

Remediation: Remediation focuses on preventing unauthorized access to sensitive configuration files by disabling directory listings and enforcing strict file permissions and access controls.

1. Identify all sensitive configuration files on the host, such as .env, config.php, or web.config, through a comprehensive audit.
2. Disable directory listing in web server configurations; for Apache, set 'Options -Indexes' in .htaccess or httpd.conf, and for Nginx, use 'autoindex off;' in server blocks.
3. Set restrictive file permissions using commands like chmod 600 or 640 to ensure only authorized users have read access.
4. Implement additional access controls such as .htaccess authentication, server-level IP restrictions, or moving files outside web-accessible directories.
5. Regularly scan the host for exposure using security tools and monitor access logs for unauthorized attempts to list or access configuration files.

#6 Low Open Redirect - Detection (severity: medium) [tags: redirect, generic, vuln]

Host: 172.19.0.3

CVSS: 6.1

Risk score: 0.435 (asset criticality 0.5)

Context: The vulnerability involves an open redirect on host 172.19.0.3, where improper URL validation allows uncontrolled redirection to external sites. Attackers could exploit this for phishing campaigns, malware distribution, or credential theft by manipulating redirect parameters in malicious links. Business impacts include potential data breaches, loss of customer trust, and regulatory penalties due to compromised user security. With a CVSS score of 6.1, this medium-severity issue poses a moderate risk requiring prompt remediation.

Exploitability rationale: The vulnerability is on an internal host with no known public exploits, reducing the likelihood of successful exploitation.

Remediation: Implement strict validation of redirect URLs to prevent unauthorized redirects by ensuring they only point to trusted destinations.

1. Audit the application code to identify all instances where HTTP redirects are performed, focusing on parameters that accept user input.
2. Define a whitelist of approved redirect targets, such as specific domains, paths, or regular expressions, based on application requirements.
3. Modify server-side code to validate redirect URLs against the whitelist before executing any redirects, rejecting invalid URLs.
4. Sanitize and encode user-supplied input used in redirect parameters to prevent injection or manipulation attacks.
5. Utilize secure redirect mechanisms provided by the application framework, such as using absolute URLs with scheme checks or built-in validation functions.
6. Conduct comprehensive testing, including automated scans and manual penetration testing, to verify that no open redirect vulnerabilities remain and redirects function as intended.

#7 Low Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves web application cookies lacking the Secure attribute, allowing them to be transmitted over unencrypted HTTP connections. Real-world attack scenarios include man-in-the-middle attacks where intercepted cookies enable session hijacking and unauthorized access. Business impacts may include data breaches, financial losses, and reputational damage due to compromised user accounts.

Exploitability rationale: The service is on an internal IP address and the vulnerability requires man-in-the-middle attacks which have higher complexity and depend on network access.

Remediation: Adding the Secure attribute to cookies ensures they are only transmitted over encrypted HTTPS connections, preventing exposure to interception attacks.

1. Audit application code and server configurations to identify all cookies set without the Secure attribute.
2. Modify the code to include the Secure flag in Set-Cookie headers for each cookie, using language-specific functions (e.g., setting cookies in web frameworks).
3. Enforce HTTPS across the entire application by redirecting HTTP traffic to HTTPS and updating server settings to disable insecure protocols.
4. Test changes in a staging environment to confirm cookies are only sent over HTTPS and that functionality remains intact.
5. Implement continuous monitoring and logging to detect any future misconfigurations or regressions.

#8 Low Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: Cookies set without the HttpOnly attribute are vulnerable to client-side script access, often due to misconfigurations in web applications. Real-world attack scenarios involve cross-site scripting (XSS) exploits where malicious JavaScript can steal session cookies, enabling session hijacking and unauthorized account access. Business impacts include data breaches, financial losses from fraud or remediation, reputational harm, and potential regulatory non-compliance penalties.

Exploitability rationale: Internal host with a misconfiguration that requires an XSS vulnerability to exploit, reducing direct exploitability.

Remediation: Add the HttpOnly attribute to cookies to prevent client-side script access and mitigate XSS risks.

1. Review application source code or configuration files to identify all instances where cookies are set.
2. Update cookie-setting code or headers to include the HttpOnly flag (e.g., Set-Cookie: name=value; HttpOnly).
3. Configure the web server or application framework to set HttpOnly by default for all cookies.
4. Test the application to verify that all cookies now contain the HttpOnly attribute in HTTP responses.
5. Implement additional security measures such as the Secure flag for HTTPS-only transmission and Content Security Policy headers.

#9 Low WAF Detection (severity: info) [tags: waf, tech, misc, discovery]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The detection of a Web Application Firewall (WAF) on the host exposes security infrastructure details to potential attackers, aiding reconnaissance efforts. In real-world attack scenarios, this information could be used to tailor attacks like SQL injection or cross-site scripting to bypass the WAF, enabling successful exploitation of protected web applications. Business impacts include data breaches, service disruptions, financial losses from incident response, and reputational damage. Additionally, it highlights risks from information leakage that could be part of broader threat actor strategies.

Exploitability rationale: WAF detection is informational, primarily aiding reconnaissance, with no public exploits, likely internal exposure, and high attack complexity for direct exploitation.

Remediation: Minimize information disclosure by securing WAF fingerprints to reduce attacker reconnaissance opportunities.

1. Consult vendor documentation for WAF hardening best practices and identify specific fingerprinting methods.
2. Configure the WAF to remove or obfuscate identifying HTTP headers, error pages, and response signatures.
3. Implement custom rules to block or limit common WAF detection probes and scanner tools.
4. Enable logging and monitoring for detection attempts to iteratively refine rules and stay updated on evasion techniques.

#10 Low Gitignore Config - Detect (severity: info) [tags: exposure, tenable, config, git, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves a misconfigured or detected .gitignore file in a Git repository, which may inadvertently expose sensitive files or configurations to version control. Attackers could exploit this by accessing exposed secrets, such as API keys or passwords, leading to unauthorized access or data breaches. This can result in compromised systems, financial loss, regulatory penalties, and damage to the organization's reputation. Properly configuring .gitignore is crucial to prevent accidental exposure of sensitive information.

Exploitability rationale: This is an internal configuration issue with no public exploits and requires access to the Git repository, which is not indicated as exposed.

Remediation: Mitigate the risk of sensitive data exposure by ensuring .gitignore files are correctly configured and enforced to exclude confidential files from version control.

1. Audit the repository to identify all .gitignore files and review their contents for missing exclusions of sensitive files such as .env, credentials, configuration files, or logs.
2. Update or create .gitignore files with appropriate patterns based on best practices, referencing vendor guidelines (e.g., GitHub's .gitignore templates) to exclude common sensitive data.
3. Use Git commands like `git rm --cached` to remove any sensitive files already tracked in the repository history.
4. Implement pre-commit hooks or integrate tools such as pre-commit framework to automatically validate .gitignore rules before each commit.
5. Provide training to developers on .gitignore best practices and establish a process for regular review and maintenance of these files.

#11 Low FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.2 is vulnerable to technology fingerprinting, which exposes detailed software and hardware information. Attackers can leverage this for reconnaissance to identify known vulnerabilities and launch targeted attacks, such as exploiting specific software flaws. The business impact includes heightened risk of security breaches, data theft, and operational disruptions, potentially leading to financial loss and reputational damage.

Exploitability rationale: Fingerprinting is an informational finding with no direct exploits or CVEs, increasing attack complexity as it requires leveraging gathered data for further attacks, and the host's private IP suggests limited exposure.

Remediation: Remediate by obscuring technology fingerprints to reduce information leakage and minimize potential reconnaissance opportunities for attackers.

1. Conduct a detailed investigation of the host (172.19.0.2) to identify the specific technology or service associated with the fingerprint, using tools like manual inspection or enhanced scanning.
2. Consult vendor documentation and security best practices for the identified technology to apply configurations that hide version information and other identifying details, such as disabling server tokens or modifying configuration files.
3. Implement additional security controls, including custom error pages, removal of unnecessary HTTP headers, and network-level restrictions like firewalls to limit exposure.
4. Establish a routine schedule for security scans and monitoring to ensure that technology fingerprints remain minimized and to detect any reoccurrence.

#12 Low Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability indicates that the host at 172.19.0.2 has cookies without the SameSite Strict attribute, a security misconfiguration that fails to enforce cross-site request restrictions. In real-world scenarios, attackers could exploit this by initiating cross-site request forgery attacks, tricking users into performing unintended actions on the vulnerable service through malicious websites. This could lead to unauthorized data access or modifications, potentially resulting in financial loss, reputational damage, and non-compliance with data protection regulations.

Exploitability rationale: The service is internal with no known public exploits, and exploiting the CSRF misconfiguration requires high attack complexity involving social engineering and network access.

Remediation: Enforce the SameSite=Strict attribute on sensitive cookies to mitigate cross-site request forgery risks by ensuring cookies are only sent in first-party contexts.

1. Audit all cookies set by the application by inspecting HTTP response headers and application code.
2. Classify cookies based on their necessity for cross-site requests; for cookies that do not require cross-site access, assign SameSite=Strict.
3. Modify the cookie-setting mechanisms in the application or server configuration to include the 'SameSite=Strict' attribute for applicable cookies.
4. Test the application in modern browsers to verify that cookies are not sent with cross-origin requests and that core functionality remains intact.
5. Deploy the changes to production, and monitor for any security alerts or functionality issues, adjusting as needed for backward compatibility with older browsers.

#13 Low Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: Technology detection via Wappalyzer indicates that host 172.19.0.2 is exposing information about its underlying technologies, such as web frameworks or libraries, which may include outdated or vulnerable components. Attackers can leverage this data for reconnaissance to identify specific weaknesses and launch targeted attacks, such as exploiting known CVEs in detected software. The business impact includes potential unauthorized access to sensitive data, service disruptions from exploits, and reputational damage due to security incidents. Mitigation involves hardening configurations to obscure technology fingerprints and ensuring all components are regularly patched.

Exploitability rationale: This finding is informational and does not directly expose a vulnerability, with low exploitability as it only aids reconnaissance and requires additional attack steps.

Remediation: Minimize exposure of technology details to reduce potential attack surfaces through hardening and updates.

1. Audit all technologies detected by Wappalyzer to identify unnecessary components and disable them.
2. Apply the latest security patches and updates for each detected technology, following vendor advisories.
3. Configure servers and applications to suppress technology identifiers, such as modifying HTTP headers and custom error pages.
4. Implement security headers (e.g., Content-Security-Policy, X-Frame-Options) and enforce HTTPS across all services.
5. Deploy a Web Application Firewall (WAF) to protect against attacks targeting known vulnerabilities in detected technologies.
6. Establish a regular schedule for vulnerability scanning and penetration testing to monitor and address new findings.

#14 Low README.md file disclosure (severity: info) [tags: exposure, markdown, files, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The README.md file is accessible on host 172.19.0.2, potentially exposing sensitive information like configuration details, internal documentation, or credentials. Real-world attack scenarios involve threat actors using this disclosure for reconnaissance to identify application weaknesses, extract valuable data, or launch targeted attacks. Business impacts include increased risk of data breaches, compliance violations, reputational damage, and financial losses if confidential information is leaked.

Exploitability rationale: The vulnerability is an information disclosure on an internal host with low attack complexity and no public exploits needed, limiting exploitability to reconnaissance.

Remediation: Restrict public access to the README.md file to prevent information leakage and reduce exposure.

1. Identify the service hosting the README.md file (e.g., web server) and locate the file's path on the host 172.19.0.2.
2. If the README.md file is not necessary for the service, delete it from the public directory.
3. If the file must be retained, move it to a non-public directory or implement server-side access controls (e.g., .htaccess rules for Apache or location blocks for Nginx).
4. Configure the service to deny direct access to sensitive files like README.md, such as by adding 'Deny from all' directives or similar configurations.
5. Verify remediation by testing access to the file from an external source and ensure it returns a 403 Forbidden or similar error.

#15 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.2 is missing critical HTTP security headers, exposing it to common web-based attacks such as Cross-Site Scripting (XSS) and clickjacking due to the absence of headers like Content-Security-Policy and X-Frame-Options. This misconfiguration can lead to data breaches, session hijacking, and regulatory non-compliance, resulting in reputational damage and financial losses. Implementing these headers is a fundamental security measure to mitigate these risks.

Exploitability rationale: The finding is on an internal IP address with no public exploits and requires higher attack complexity like client-side attacks for exploitation.

Remediation: Implement missing HTTP security headers to enhance web application security and mitigate common vulnerabilities.

1. Identify missing security headers by using tools like curl or browser developer tools to inspect HTTP response headers.
2. Research and select appropriate security headers (e.g., Content-Security-Policy, X-Frame-Options, Strict-Transport-Security) based on OWASP guidelines and application requirements.
3. Configure the web server or application (e.g., via server configuration files or middleware) to include the necessary headers in all HTTP responses.
4. Apply the configuration changes and restart or reload the web server to ensure they take effect.
5. Test the changes by verifying headers are present in responses and checking for any functionality issues, then set up ongoing monitoring to maintain compliance.

#16 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The web service at host 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy and X-Frame-Options, indicating a misconfiguration that leaves it vulnerable to client-side attacks. Real-world scenarios include clickjacking, where attackers embed the site in malicious iframes to deceive users, and MIME-type sniffing that could enable cross-site scripting (XSS). Business impacts may involve data breaches, compromised user sessions, financial losses from fraud, and reputational harm due to security lapses.

Exploitability rationale: The misconfiguration requires user interaction or additional vulnerabilities for exploitation, with high attack complexity and no direct public exploits.

Remediation: Add appropriate HTTP security headers to responses to mitigate common web vulnerabilities such as XSS and clickjacking.

1. Use HTTP header testing tools (e.g., curl, browser developer tools, or security scanners) to identify which security headers are currently missing.
2. Consult vendor advisories and best practice guidelines, such as the OWASP Secure Headers Project, to determine recommended headers (e.g., Content-Security-Policy, X-Frame-Options, X-Content-Type-Options) and their values.
3. Configure the web server or application framework to include the missing security headers in all HTTP responses, adjusting settings based on the specific server technology (e.g., Apache, Nginx, IIS).
4. Apply configuration changes and restart the web server or application services to ensure the new headers are active.
5. Test the implementation by verifying HTTP responses and ensuring the headers do not break application functionality or user experience.
6. Establish a routine audit process to monitor and update security headers periodically, incorporating new best practices and threat intelligence.

#17 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The HTTP service at 172.19.0.2 is missing critical security headers, which are essential for protecting against common web-based attacks. This misconfiguration could enable real-world exploits such as cross-site scripting (XSS), clickjacking, and data sniffing, potentially compromising user sessions and sensitive information. From a business perspective, this vulnerability increases the risk of data breaches, regulatory non-compliance, and damage to organizational reputation.

Exploitability rationale: The service is internally exposed, lacks direct public exploits, and exploiting missing security headers requires additional vulnerabilities, resulting in low exploitability.

Remediation: Remediate by adding recommended HTTP security headers to the server configuration to prevent common web vulnerabilities and enhance security posture.

1. Identify the web server software (e.g., Apache, Nginx, IIS) or application framework hosting the service at 172.19.0.2.
2. Consult vendor documentation or security best practices (e.g., OWASP) for essential headers such as X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Content-Security-Policy, and Referrer-Policy.
3. Configure the server to include these headers in HTTP responses by modifying server configuration files (e.g., httpd.conf, nginx.conf), .htaccess files, or application code.
4. Test the configuration to verify headers are correctly set using tools like curl with the '-I' flag or online header analysis services.
5. Implement monitoring or automation to ensure headers remain consistently applied and updated according to evolving security standards.

#18 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.2 is missing essential HTTP security headers, such as Content-Security-Policy and X-Frame-Options. This misconfiguration can expose the application to attacks like cross-site scripting (XSS) and clickjacking, allowing attackers to hijack user sessions or steal sensitive information. In a business context, this could lead to data breaches, loss of customer trust, and potential regulatory fines due to non-compliance with security standards.

Exploitability rationale: Internal HTTP service with no direct public exploits for misconfigurations and limited exposure, requiring additional attack vectors to leverage.

Remediation: Implement HTTP security headers to mitigate common web vulnerabilities and enhance overall application security.

1. Identify the web server software or application framework in use on host 172.19.0.2 (e.g., Apache, Nginx, IIS, or a programming language runtime).
2. Use tools like curl or browser developer tools to inspect current HTTP response headers and confirm which standard security headers are missing.
3. Refer to vendor documentation and security best practices (e.g., OWASP, Mozilla Web Security) to determine appropriate headers, such as Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security if HTTPS is enabled.
4. Configure the web server or application settings to add the missing headers; for example, modify configuration files (e.g., .htaccess, nginx.conf) or code to set header values.
5. Test the changes by sending HTTP requests to the host and verifying that the security headers are correctly included in responses without causing functional issues.
6. Implement monitoring or periodic scans to ensure headers remain properly configured over time and update them as needed based on evolving security standards.

#19 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The web server at 172.19.0.2 is missing HTTP security headers in its responses, which are essential defenses against common web attacks. This misconfiguration can lead to real-world attack scenarios such as clickjacking, cross-site scripting, or man-in-the-middle attacks if headers like X-Frame-Options or Strict-Transport-Security are absent. The business impact includes increased vulnerability to data breaches, potential non-compliance with regulatory standards, and reputational harm due to security incidents.

Exploitability rationale: This is an internal misconfiguration with no known exploits, requiring complex attack scenarios like social engineering or MITM conditions.

Remediation: Implement standard HTTP security headers to mitigate common web vulnerabilities and enhance the application's security posture.

1. Identify the web server or application software running on the host (e.g., Apache, Nginx, IIS, or a custom framework) to determine the appropriate configuration method.
2. Review current HTTP response headers using tools such as curl, browser developer tools, or security scanners to confirm which headers are missing (e.g., Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security).
3. Configure the identified web server or application to add the missing security headers based on vendor documentation and industry best practices, ensuring headers are set globally for all responses.
4. Test the configuration by sending HTTP requests to the host and verifying that the security headers are correctly applied, including on error pages and different resource types.
5. Monitor the application for any functionality issues post-implementation and adjust header values as needed to balance security and usability.
6. Document the changes made, update security policies, and schedule periodic reviews to maintain compliance with evolving standards.

#20 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The HTTP service on host 172.19.0.2 lacks essential security headers, such as Content Security Policy and X-Frame-Options, leaving it vulnerable to misconfiguration-based attacks. Real-world scenarios include cross-site scripting (XSS) for injecting malicious scripts or clickjacking to trick users into unintended actions, potentially leading to session hijacking or data theft. The business impact involves risks of data breaches, compliance violations, and reputational harm due to unauthorized access and compromised user trust.

Exploitability rationale: Internal service exposure with no direct public exploits and high attack complexity for exploiting missing headers, which require additional vulnerabilities or user interaction.

Remediation: Add missing HTTP security headers to enhance web application security and protect against common attacks.

1. Identify the web server or application type by scanning the service and reviewing configuration files.
2. Refer to OWASP Secure Headers Project and vendor-specific documentation for recommended security headers.
3. Configure the server to include headers like Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security.
4. Apply changes and restart the web service if required, ensuring no downtime or service disruption.
5. Test the implementation using HTTP header analysis tools (e.g., curl, browser dev tools) to confirm headers are correctly set.
6. Establish monitoring and regular reviews to maintain header compliance and update as needed based on evolving threats.

#21 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The web service at host 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy or X-Frame-Options, which are essential for mitigating web-based attacks. This misconfiguration could enable real-world scenarios like Cross-Site Scripting or Clickjacking, where attackers exploit the lack of protections to steal data or compromise user interactions. From a business perspective, while the severity is informational, it poses risks including data breaches, reputational damage, and non-compliance with security regulations.

Exploitability rationale: The service is internal-only with no direct public exploits, and exploiting missing headers requires additional application-level vulnerabilities, increasing attack complexity.

Remediation: Implement missing HTTP security headers to enhance the application's defense against common web attacks like cross-site scripting and clickjacking.

1. Identify the web server or application framework (e.g., Apache, Nginx, IIS, Node.js) by reviewing server configurations or application code.
2. Research vendor-specific documentation for adding security headers, such as from OWASP or the server's official guides, to ensure best practices are followed.
3. Configure the server or application to include key security headers in HTTP responses, starting with Strict-Transport-Security (HSTS), Content-Security-Policy (CSP), X-Content-Type-Options, and X-Frame-Options.
4. Test the implementation using browser developer tools or online header-checking tools to verify headers are correctly applied and do not disrupt functionality.
5. Integrate header validation into regular security scans or CI/CD pipelines to monitor compliance and address any future misconfigurations.

#22 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy and X-Frame-Options, which are essential for protecting web applications from common threats. Attackers can exploit this vulnerability to launch cross-site scripting (XSS) attacks, clickjacking, or content sniffing, potentially stealing sensitive data or hijacking user sessions. Business impact includes increased risk of data breaches, financial losses, reputational harm, and non-compliance with security standards or regulations.

Exploitability rationale: The service is internal and missing security headers are a misconfiguration that requires additional vulnerabilities or user interaction to exploit, with no direct public exploits available.

Remediation: Implement standard HTTP security headers to protect against web vulnerabilities like clickjacking, content sniffing, and injection attacks.

1. Scan HTTP responses using tools like curl or browser developer tools to identify which security headers are missing.
2. Configure the web server or application to add recommended security headers such as Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY or SAMEORIGIN, and Strict-Transport-Security.
3. Apply the changes in the server configuration file (e.g., Apache's .htaccess or Nginx's nginx.conf) or application middleware.
4. Test the updated HTTP responses to ensure headers are correctly set without breaking functionality, using automated scanning or manual verification.
5. Document the implementation and establish a routine for reviewing and updating security headers based on evolving best practices and threats.

#23 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The web service on host 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy and X-Frame-Options, which are essential for protecting against client-side attacks. This misconfiguration can enable real-world attacks like cross-site scripting (XSS), clickjacking, or content sniffing, allowing attackers to inject malicious code or manipulate user interactions. Successful exploitation could lead to data theft, session hijacking, or unauthorized actions, resulting in financial loss, regulatory penalties, and reputational damage for the business.

Exploitability rationale: Internal service with no known exploits and requires complex attack chains involving other vulnerabilities to exploit the misconfiguration.

Remediation: Implement and configure standard HTTP security headers to protect against common web vulnerabilities like cross-site scripting and clickjacking, following best practices from sources such as OWASP.

1. Audit current HTTP response headers using tools like curl or browser developer tools to identify which security headers are missing.
2. Research recommended security headers from vendor advisories or standards (e.g., OWASP Secure Headers Project) such as Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, and Referrer-Policy.
3. Configure the web server or application framework to add the missing headers; for example, in Apache use 'Header set' directives, in Nginx use 'add_header', or in application code use middleware or plugins.
4. Test the application thoroughly to ensure the new headers are correctly applied without breaking functionality, using automated scanning tools or manual checks.
5. Establish a process for regular review and maintenance of security headers during updates or deployments to keep configurations aligned with evolving best practices.

#24 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The web server at 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy or X-Frame-Options, in its responses. This misconfiguration exposes it to real-world attacks like cross-site scripting (XSS) and clickjacking, potentially allowing attackers to inject malicious scripts or hijack user sessions. The business impact includes increased risk of data breaches, compliance violations, reputational damage, and financial losses.

Exploitability rationale: The service is internal with no direct exploits available, and exploiting the misconfiguration requires additional vulnerabilities or complex attack chains.

Remediation: Implement HTTP security headers to mitigate risks from missing protective measures in web responses, following vendor guidelines and security best practices.

1. Identify the web server software or application framework running on host 172.19.0.2 by checking service banners or configuration files.
2. Consult vendor-specific documentation (e.g., for Apache, Nginx, or IIS) or trusted security sources like OWASP for recommended HTTP security headers.
3. Configure the missing security headers in the server or application configuration, prioritizing Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security (if HTTPS is used), and Referrer-Policy.
4. Test the implementation by sending HTTP requests to the host and verifying headers are present and correctly set using tools like curl or browser developer tools.
5. Document the changes and establish a routine for reviewing and updating security headers during system updates or configuration changes.

#25 Low **robots.txt endpoint prober (severity: info) [tags: miscellaneous, misc, generic, discovery]**

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves probing for the robots.txt file, a standard web configuration used to guide crawler behavior. Attackers may use this reconnaissance to discover hidden directories or sensitive paths, aiding in planning targeted attacks like data exfiltration or unauthorized access. While directly low-impact, this information disclosure can lead to increased risk of security breaches if not monitored, potentially affecting confidentiality and operational integrity.

Exploitability rationale: The service is likely internal, there are no known public exploits, attack complexity is low, and it is an informational finding without direct exploit potential.

Remediation: Remediate by securing the robots.txt file to prevent information disclosure and enhancing monitoring to detect reconnaissance attempts.

1. Review and edit the robots.txt file to remove any sensitive or unnecessary entries that could expose internal paths or confidential directories.
2. Configure the web server to return a 403 Forbidden or 404 Not Found response for invalid or suspicious requests to the robots.txt endpoint.
3. Implement logging for all access to robots.txt and set up alerts for unusual patterns, such as repeated or automated probes.
4. Deploy a web application firewall (WAF) or intrusion detection system (IDS) to monitor and block malicious probing activities.

#26 Low robots.txt file (severity: info) [tags: miscellaneous, misc, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 (asset criticality 0.5)

Context: An exposed robots.txt file on host 172.19.0.2 can disclose hidden directories or sensitive paths to attackers. This information can be leveraged in real-world attack scenarios for reconnaissance, aiding in the discovery of high-value targets like admin interfaces or backup files. Although the severity is informational, the business impact is low, but it may expose internal structures that could be exploited in more targeted attacks.

Exploitability rationale: This is an informational finding with no direct exploitability, as it only aids reconnaissance and is likely internal with low attack complexity and no known exploits.

Remediation: Remediate the robots.txt file to prevent information disclosure of sensitive paths by reviewing and securing its contents based on web security best practices.

1. Locate and access the robots.txt file at the root directory of the host 172.19.0.2.
2. Review the file for any entries that expose sensitive directories or files, such as admin or internal paths.
3. Update the robots.txt file to use 'Disallow' directives only for intended crawl restrictions, removing or obfuscating sensitive information.
4. Configure the web server to restrict access to the robots.txt file if it contains confidential data.
5. Implement regular audits and updates of the robots.txt file as part of ongoing security maintenance.

#27 Low Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: HTTP cookies on host 172.19.0.3 lack the Secure attribute, allowing them to be transmitted over unencrypted HTTP connections. Attackers could exploit this through man-in-the-middle attacks on insecure networks to intercept cookies and steal session data, leading to unauthorized access. This vulnerability poses risks of data breaches, reputational damage, and potential regulatory non-compliance for the business.

Exploitability rationale: Internal service with misconfiguration requiring man-in-the-middle attacks on the network, which increases attack complexity and limits exposure.

Remediation: Set the Secure attribute on cookies to ensure they are only transmitted over HTTPS, preventing exposure in unencrypted traffic.

1. Identify all cookie-setting points in the application code, server configuration, or frameworks.
2. Modify each cookie definition to include the Secure attribute, such as by setting 'Secure' flag in Set-Cookie headers or using platform-specific parameters (e.g., 'Secure=true' in code).
3. Test the application to verify that cookies are not sent over HTTP and that functionality remains intact over HTTPS.
4. Review and set additional security attributes like HttpOnly and SameSite to enhance overall cookie security.
5. Implement automated code reviews or security testing tools to enforce the Secure attribute on all future cookie implementations.

#28 Low Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: Cookies on the host 172.19.0.3 lack the HttpOnly attribute, exposing them to client-side script access. This vulnerability can be leveraged in cross-site scripting (XSS) attacks to steal session tokens or other sensitive cookie data. Consequently, attackers may gain unauthorized access to user sessions, leading to potential data breaches and loss of confidentiality.

Exploitability rationale: This misconfiguration requires a separate XSS vulnerability to be exploited and is on an internal host, limiting exposure and increasing attack complexity.

Remediation: Enable HttpOnly attribute on cookies to prevent client-side script access and mitigate cross-site scripting risks.

1. Audit all application cookies to identify those without HttpOnly attribute through code review or configuration inspection.
2. Modify application code or server settings to include the HttpOnly flag when setting cookies (e.g., in PHP use `setcookie()` with `httponly` parameter).
3. Test changes in a staging environment to ensure cookies are correctly set with HttpOnly and application functionality remains intact.
4. Deploy updates to production and validate using security scanning tools to confirm HttpOnly is applied.

#29 Low SNMPv3 Fingerprint - Detect (severity: info) [tags: js, udp, network, snmp]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability finding indicates the presence of SNMPv3 on host 172.19.0.3, which is a network management protocol that could be misconfigured or have inherent weaknesses. Real-world attack scenarios include brute-force attacks on authentication credentials, exploitation of known SNMP vulnerabilities, or using the service for reconnaissance to map network devices. The business impact involves risks such as unauthorized access to network configurations, sensitive data exposure, or potential denial-of-service disruptions affecting operational continuity.

Exploitability rationale: SNMPv3 on a private IP address with no known CVEs or specified internet exposure, and authentication requirements increase attack complexity.

Remediation: Secure SNMPv3 services by enforcing strong authentication, encryption, and access controls to mitigate unauthorized access risks.

1. Assess the necessity of the SNMPv3 service; if not required, disable it entirely on the host.
2. If SNMPv3 is needed, reconfigure it to use strong security protocols such as SHA-256 for authentication and AES-256 for encryption, as per vendor recommendations.
3. Implement network-level access controls, such as firewalls or ACLs, to restrict SNMP access to only authorized management stations.
4. Regularly update and patch the SNMP software to address any emerging vulnerabilities and ensure compliance with security best practices.
5. Enable logging and monitoring for SNMP traffic to detect and respond to suspicious activities promptly.

#30 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The HTTP service on host 172.19.0.3 is missing critical security headers, such as Content-Security-Policy and X-Frame-Options, indicating a misconfiguration. This vulnerability exposes the service to real-world attacks like Cross-Site Scripting (XSS) for code injection and Clickjacking for unauthorized user actions, potentially leading to data theft or session hijacking. Business impacts include compromised data integrity, loss of customer trust, and regulatory non-compliance resulting in fines or reputational damage.

Exploitability rationale: The service is internal with no public exploits available, and exploiting missing security headers requires high attack complexity and additional vulnerabilities.

Remediation: To mitigate risks from missing HTTP security headers, implement standard headers as per OWASP guidelines to protect against common web vulnerabilities.

1. Use HTTP inspection tools (e.g., curl, browser dev tools) to audit current response headers and identify missing ones.
2. Consult OWASP Secure Headers Project or vendor advisories (e.g., Apache, Nginx documentation) for recommended security headers such as Content-Security-Policy, X-Content-Type-Options, and X-Frame-Options.
3. Configure the web server or application to add the missing headers; for example, edit configuration files (e.g., httpd.conf for Apache, nginx.conf for Nginx) or implement via middleware in application code.
4. Test the changes in a non-production environment to ensure headers are correctly applied without breaking functionality.
5. Deploy the configuration to production and verify headers are present using HTTP requests.
6. Document the remediation and establish a process for periodic reviews to maintain header security.

#31 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The server at 172.19.0.3 is missing HTTP security headers, which are essential for mitigating web-based attacks such as cross-site scripting and clickjacking. This misconfiguration could enable attackers to steal user data, hijack sessions, or perform phishing attacks by exploiting the lack of protections like Content-Security-Policy or X-Frame-Options. Business impacts include increased risk of data breaches, loss of customer trust, financial penalties from regulatory non-compliance, and potential reputational damage.

Exploitability rationale: The service is internal with no direct exploits, and exploiting the misconfiguration requires additional vulnerabilities, making it low risk.

Remediation: Implement standard HTTP security headers to reduce exposure to common web vulnerabilities and improve security posture.

1. Identify the web server software (e.g., Apache, Nginx) on host 172.19.0.3.
2. Review current HTTP responses for missing headers and consult vendor documentation or best practices (e.g., OWASP guidelines).
3. Configure the web server to add key security headers such as Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, and Referrer-Policy.
4. Test the configuration changes in a non-production environment to ensure headers are correctly applied without breaking functionality.
5. Deploy the updated configuration to production and verify headers using tools like curl or browser developer tools.

#32 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The HTTP service on host 172.19.0.3 is missing standard security headers, making it susceptible to misconfiguration-related vulnerabilities. Real-world attack scenarios include cross-site scripting (XSS), clickjacking, or data leakage due to absent headers like Content-Security-Policy or X-Frame-Options. This could lead to unauthorized access, session hijacking, or compromised user data, impacting business reputation and potentially causing financial losses.

Exploitability rationale: The service is internal (private IP), missing headers are informational with no direct exploits, and exploitation requires additional user interaction or vulnerabilities, increasing complexity.

Remediation: Implement missing HTTP security headers to protect against common web vulnerabilities like clickjacking, content sniffing, and protocol downgrade attacks, adhering to security best practices.

1. Audit current HTTP responses on host 172.19.0.3 using tools like curl (e.g., 'curl -I') or browser developer tools to identify which security headers are missing.
2. Determine the web server or application framework (e.g., Apache, Nginx, IIS, or specific software) to tailor configuration appropriately.
3. Add missing headers based on vendor advisories and best practices: set 'X-Content-Type-Options: nosniff', 'X-Frame-Options: DENY' or 'SAMEORIGIN', and create a 'Content-Security-Policy' header with a restrictive policy.
4. If HTTPS is in use, enable 'Strict-Transport-Security' (HSTS) with a 'max-age' directive (e.g., 31536000 seconds) and include subdomains if applicable.
5. Update the server or application configuration (e.g., via .htaccess for Apache, nginx.conf for Nginx, or application code) and restart or reload the service to apply changes.
6. Verify the implementation by re-checking HTTP responses to confirm headers are correctly set and ensure no functional regressions occur.
7. Document the security header configuration and integrate it into continuous monitoring and regular security reviews to maintain compliance.

#33 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.3 has HTTP services missing critical security headers like Content-Security-Policy and X-Frame-Options, which are fundamental protections against common web attacks. Attackers could exploit this misconfiguration to perform clickjacking, cross-site scripting, or data theft, compromising user sessions and sensitive information. This could lead to data breaches, regulatory fines, financial losses, and reputational damage for the organization.

Exploitability rationale: The service is on an internal IP with no public exploits, requiring network access to exploit the misconfiguration.

Remediation: Implement standard HTTP security headers to mitigate common web vulnerabilities such as XSS and clickjacking.

1. Audit current HTTP response headers to identify which security headers are missing.
2. Refer to OWASP Secure Headers Project or vendor guidelines to determine recommended headers like Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security.
3. Configure the web server or application code to add or enable the missing security headers in all HTTP responses.
4. Test the changes in a staging environment to verify headers are correctly applied and do not disrupt functionality.
5. Deploy to production and establish monitoring to ensure headers remain properly configured and updated per security best practices.

#34 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The HTTP service on host 172.19.0.3 lacks essential security headers, indicating a misconfiguration that exposes it to common web-based threats. Real-world attack scenarios include cross-site scripting (XSS), clickjacking, and information leakage due to inadequate protections like Content-Security-Policy or X-Frame-Options headers. Business impacts may involve data breaches, regulatory non-compliance, financial losses from incident response, and reputational damage from perceived security weaknesses.

Exploitability rationale: The internal IP suggests limited exposure, and missing security headers alone without specific exploits or version details result in low exploitability requiring additional attack conditions.

Remediation: Implement missing HTTP security headers to mitigate common web vulnerabilities and enhance server security.

1. Identify the web server software and version on the host 172.19.0.3 to determine the appropriate configuration method.
2. Consult security best practices (e.g., OWASP guidelines) to list recommended security headers, such as Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Strict-Transport-Security.
3. Modify the web server configuration file (e.g., httpd.conf for Apache, nginx.conf for Nginx) to add the missing headers based on vendor documentation.
4. Restart or reload the web server service to apply the configuration changes.
5. Verify the implementation by checking HTTP response headers using tools like curl or browser developer tools to ensure headers are correctly set.

#35 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The web service on host 172.19.0.3 is missing critical HTTP security headers, such as Content-Security-Policy or X-Frame-Options, which are essential for protecting against common web vulnerabilities. Real-world attack scenarios include cross-site scripting (XSS) through script injection, clickjacking that can hijack user interactions, or data leakage via insecure referrer policies. The business impact involves potential compromise of user data, website defacement, and reputational damage, leading to financial losses and regulatory non-compliance.

Exploitability rationale: Internal service with missing security headers, which are misconfigurations that require additional vulnerabilities or client-side factors to exploit, resulting in low direct exploitability.

Remediation: Enhance web security by configuring the web server to include standard HTTP security headers to mitigate risks from common attacks like XSS and clickjacking.

1. Identify the web server software and version running on host 172.19.0.3 by checking service banners or configuration files.
2. Consult vendor-specific documentation or OWASP Secure Headers Project guidelines to determine appropriate headers for your server.
3. Configure the server to add headers such as Content-Security-Policy: default-src 'self';, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin, and Strict-Transport-Security: max-age=31536000; includeSubDomains if HTTPS is enforced.
4. Apply the configuration changes by editing server files (e.g., httpd.conf for Apache or nginx.conf for Nginx) and restart the web server service.
5. Verify the headers are present in HTTP responses using tools like curl -I or browser developer tools to ensure proper implementation.

#36 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.3 is missing critical HTTP security headers, such as Content Security Policy and X-Frame-Options, indicating a web service misconfiguration. Real-world attack scenarios include clickjacking, where attackers embed the site in malicious frames to trick users, or cross-site scripting (XSS) by exploiting the lack of CSP to inject malicious scripts. Business impacts involve potential data breaches, unauthorized access to sensitive information, and reputational damage from security incidents, leading to financial losses and regulatory penalties.

Exploitability rationale: The service is on an internal IP address with no known public exploits, and exploitation requires additional attack vectors such as user interaction.

Remediation: Implement missing HTTP security headers to mitigate common web vulnerabilities and align with security best practices.

1. Audit HTTP response headers using tools like curl or browser developer tools to identify which security headers (e.g., Content-Security-Policy, X-Content-Type-Options, X-Frame-Options) are missing.
2. Refer to trusted guidelines such as the OWASP Secure Headers Project to define appropriate header values based on the application's needs and security requirements.
3. Configure the web server or application framework to add the missing headers, for example by editing configuration files (e.g., .htaccess for Apache, server blocks for Nginx) or using middleware in the application code.
4. Test the updated configuration in a staging environment to verify that headers are correctly applied and that functionality remains unaffected.
5. Document the changes, implement continuous monitoring for header presence, and schedule periodic reviews to ensure ongoing compliance.

#37 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The host at 172.19.0.3 is missing critical HTTP security headers, such as Content-Security-Policy or X-Frame-Options, which are essential for securing web communications. This misconfiguration could enable real-world attacks like clickjacking, where malicious sites trick users into unintended actions, or cross-site scripting (XSS) that allows attackers to inject harmful scripts. The business impact includes potential data breaches, loss of customer trust, and non-compliance with security standards, leading to financial and reputational harm.

Exploitability rationale: Internal service exposure with no public exploits and high attack complexity for leveraging missing headers.

Remediation: Implementing standard HTTP security headers in the web server configuration mitigates risks from clickjacking, MIME sniffing, and cross-site scripting by enforcing protective policies.

1. Identify the web server software (e.g., Apache, Nginx) and its configuration file on host 172.19.0.3 by checking common paths like `/etc/nginx/` or `/etc/apache2/`.
2. Edit the configuration file to add headers: 'X-Content-Type-Options: nosniff', 'X-Frame-Options: DENY', 'Referrer-Policy: no-referrer', and 'Content-Security-Policy: default-src 'self'; script-src 'self"' (adjust CSP as needed for application requirements).
3. If HTTPS is configured, add 'Strict-Transport-Security: max-age=31536000; includeSubDomains' to enforce secure connections.
4. Save the configuration file and restart the web server service (e.g., `systemctl restart nginx` or `apache2`) to apply changes.
5. Verify the headers by using tools like `curl -I http://172.19.0.3` or browser developer tools to inspect HTTP response headers.

#38 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves a web service at 172.19.0.3 missing standard HTTP security headers like Content-Security-Policy and X-Frame-Options, which are critical for mitigating client-side attacks. This misconfiguration enables real-world scenarios such as clickjacking, cross-site scripting, and data injection, allowing attackers to compromise user sessions or inject malicious content. The business impact includes potential data breaches, reputational damage, and regulatory non-compliance, leading to financial losses and legal penalties.

Exploitability rationale: The service is internal, lacks known public exploits, and requires client-side interaction for exploitation, limiting its practical exploitability.

Remediation: Implement missing HTTP security headers to enhance web application security and protect against common vulnerabilities like clickjacking, content sniffing, and insecure data transfers.

1. Identify specific missing security headers by inspecting HTTP response headers using tools like curl -I, browser developer tools, or security scanners.
2. Research best practices from OWASP or vendor documentation to define appropriate header values, focusing on common headers like Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security.
3. Configure the web server (e.g., Apache, Nginx) or application framework to add these headers to all HTTP responses, ensuring compatibility with the application's functionality.
4. Test the implementation in a staging environment to verify headers are correctly applied and do not disrupt user experience or services.
5. Deploy the configuration to production and establish regular audits or automated checks to monitor header compliance and update values as security standards evolve.

#39 Low FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The host is exposing technology fingerprint information via FingerprintHub, revealing details about its software stack and configurations. Attackers can use this information to identify and exploit known vulnerabilities in the detected technologies, leading to potential unauthorized access or system compromise. This could result in data breaches, service disruptions, and financial losses. Additionally, it may damage the organization's reputation and lead to compliance violations.

Exploitability rationale: The internal service with no direct exploits available requires additional steps for exploitation, limiting immediate risk.

Remediation: Minimize information exposure and potential attack surface by reducing technology fingerprinting details.

1. Identify the specific technology or service on host 172.19.0.3 by reviewing system configurations, logs, or using reconnaissance tools.
2. Apply vendor-recommended hardening configurations to suppress version information and unnecessary banners, such as hiding 'Server' headers in web servers.
3. Implement network segmentation or access control lists to limit exposure of the host from untrusted networks.
4. Document the finding and incorporate it into routine security assessments to monitor for changes or new fingerprinting risks.

#40 Low Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves the exposure of web technology information through Wappalyzer detection on host 172.19.0.3, which reveals details about the software stack in use. Attackers can use this data for reconnaissance to identify potential weaknesses in specific technologies and craft targeted attacks. This increases the risk of exploits, such as leveraging known vulnerabilities in detected frameworks or servers. The business impact includes heightened exposure to cyber threats, potential data breaches, and reputational harm from unauthorized access or service disruptions.

Exploitability rationale: This is an informational finding with no direct exploit; exploitation depends on subsequent vulnerabilities, and the service is on an internal IP with no specified public exploits.

Remediation: Reduce information disclosure and attack surface by securing or hiding technology identifiers and ensuring detected components are patched and configured according to best practices.

1. Identify all technologies detected by Wappalyzer on host 172.19.0.3 through a detailed scan or manual review.
2. For each technology, check vendor advisories for security updates and apply the latest patches to mitigate known vulnerabilities.
3. Configure services to suppress technology fingerprints, such as disabling version numbers in server headers (e.g., in Apache, set 'ServerTokens Prod'; in Nginx, use 'server_tokens off;').
4. Implement security headers like Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options to reduce risks from detected technologies.
5. Establish regular vulnerability scans and monitoring to track technology changes and ensure ongoing compliance with security updates.

#41 Low Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: This informational finding indicates that the host's technology stack is being detected by Wappalyzer, exposing details about software and frameworks in use. Attackers can leverage this information for reconnaissance to identify specific technologies and target known vulnerabilities, such as in outdated versions of web applications. This increases the risk of successful exploitation, potentially leading to unauthorized access, data breaches, and operational disruptions for the business.

Exploitability rationale: This is an informational reconnaissance finding with no direct exploit, low service exposure, and no known vulnerabilities.

Remediation: Reduce information leakage by obscuring technology detection through server configuration and header manipulation.

1. Review and modify web server configuration files to disable display of version and technology-specific headers (e.g., set 'ServerTokens Prod' and 'ServerSignature Off' in Apache, or 'server_tokens off;' in Nginx).
2. Remove or alter 'X-Powered-By' and similar headers in application code, frameworks, or server middleware.
3. Deploy a reverse proxy or web application firewall (WAF) to mask backend technologies and enforce security headers like 'Server' and 'X-Powered-By' stripping.
4. Conduct regular scans with technology detection tools (e.g., Wappalyzer) to verify that information leakage is minimized and configurations remain effective.

#42 Low Wordpress Plugin Detection (severity: info) [tags: tech, wp, wordpress, plugin, detection, wp-plugin, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability involves the detection of WordPress plugins, which exposes information about the site's software stack and installed components. Real-world attack scenarios include attackers using this data to identify and target plugins with known vulnerabilities, potentially leading to exploitation through methods like SQL injection or remote code execution. The business impact could range from minor disruptions to significant data breaches, reputational damage, and financial losses, especially if sensitive user data is compromised or the site is defaced.

Exploitability rationale: This is an informational finding with no direct vulnerability or public exploits, and the host is on an internal network, reducing exposure and immediate risk.

Remediation: Reduce exposure by securing WordPress plugin management and hardening the site to prevent unnecessary information disclosure.

1. Log in to the WordPress admin dashboard and update all plugins to their latest versions via the Updates section.
2. Deactivate and delete any unused or inactive plugins to minimize the attack surface.
3. Edit the .htaccess file in the WordPress root directory to restrict access to plugin directories, for example by adding 'Options -Indexes' or denying access to wp-content/plugins.
4. Install and configure a reputable security plugin (e.g., Wordfence or iThemes Security) to hide plugin details and implement additional hardening measures.
5. Establish a regular audit schedule to monitor for plugin updates and apply security patches promptly based on vendor advisories.

#43 Low Wordpress Plugin Detection (severity: info) [tags: tech, wp, wordpress, plugin, detection, wp-plugin, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The detection of a WordPress plugin on host 172.19.0.3 indicates potential exposure to vulnerabilities in third-party components. Attackers often exploit outdated or insecure plugins through scenarios like remote code execution or data injection to compromise websites. This could lead to data breaches, service disruptions, and reputational damage for the organization.

Exploitability rationale: Internal service with no specific vulnerabilities or public exploits identified, and the detection is informational with high attack complexity due to lack of details.

Remediation: Remediation focuses on securing WordPress plugin deployments by enforcing updates, removing unnecessary plugins, and implementing security hardening measures.

1. Audit all active WordPress plugins on the host and document their versions.
2. Update all plugins to the latest stable releases from the official WordPress repository or trusted sources.
3. Deactivate and delete any plugins that are unused, outdated, or have known security vulnerabilities.
4. Install and configure a reputable WordPress security plugin (e.g., Wordfence or Sucuri Security) to monitor and protect against threats.
5. Establish a regular schedule for plugin security scans and updates, and enable notifications for new vulnerabilities.

#44 Low Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The host's cookies are missing the SameSite Strict attribute, which could leave them vulnerable to cross-site request forgery (CSRF) attacks. In a real-world scenario, attackers might exploit this by sending malicious requests from other sites using the victim's cookies to perform unauthorized actions on their behalf. This could lead to unauthorized transactions, data breaches, or account compromises. The business impact includes potential financial losses, reputational damage, and loss of customer trust due to security weaknesses.

Exploitability rationale: The vulnerability is on an internal host with no public exploits and requires social engineering for exploitation.

Remediation: Add the SameSite=Strict attribute to cookies to mitigate cross-site request forgery risks and enhance web application security.

1. Identify all cookies set by the application or service, particularly those missing the SameSite attribute, by reviewing HTTP response headers or application code.
2. Modify the cookie configuration to include the 'SameSite=Strict' attribute in the Set-Cookie header for all relevant cookies, following vendor-specific guidance such as OWASP recommendations or framework documentation.
3. Test the application in a controlled environment to ensure that the SameSite attribute does not break functionality, especially for cross-origin requests that may require adjustment.
4. Deploy the updated configuration to production, starting with a canary release if possible to minimize risk.
5. Implement monitoring for any security or functionality issues related to cookie handling post-remediation, and iterate as needed.

#45 Low FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability finding indicates that the host at 172.19.0.3 is being fingerprinted by FingerprintHub technology, revealing details about the technologies in use. This information can be leveraged by attackers for reconnaissance to identify specific software versions or configurations that may have known vulnerabilities. In real-world scenarios, this could facilitate targeted attacks, such as exploiting outdated components or misconfigurations. Business impact includes an elevated risk of cyber incidents, potential data exposure, and reputational damage if the fingerprinted technologies are associated with sensitive operations.

Exploitability rationale: This is an internal, informational fingerprinting finding with no associated CVEs, public exploits, or direct attack vector, reducing exploitability.

Remediation: To minimize reconnaissance risks by identifying and restricting exposure of technology details.

1. Identify the specific service or technology on host 172.19.0.3 using tools like Nmap or service discovery scripts.
2. Review and tighten network firewall rules to limit inbound access to only essential ports and services.
3. Configure the service to disable or modify default banners, headers, and responses that leak technology information.
4. Implement network segmentation to isolate the host and reduce exposure to untrusted networks.
5. Schedule regular vulnerability scans and security audits to monitor for similar findings and ensure ongoing compliance.

#46 Low Semantic UI Framework - Detect (severity: info) [tags: tech, semantic-ui, discovery]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The detection of the Semantic UI Framework on host 172.19.0.3 indicates its presence in the web application technology stack, which could be a target for reconnaissance. Attackers might exploit known vulnerabilities in Semantic UI, such as cross-site scripting (XSS) flaws, to inject malicious scripts and compromise user sessions or data. If unpatched, this could lead to data breaches, service disruption, and reputational harm, affecting business integrity and customer trust.

Exploitability rationale: Internal network exposure with no confirmed unpatched vulnerabilities or known public exploits, limiting attack surface.

Remediation: Maintain the Semantic UI framework by updating to the latest version and following secure configuration practices to mitigate potential future risks.

1. Identify the current version of Semantic UI in use by checking package files or build scripts.
2. Visit the official Semantic UI GitHub repository or website to determine the latest stable release.
3. Update Semantic UI to the latest version using a package manager like npm or yarn, following vendor upgrade instructions.
4. Review the application code to ensure no insecure usage of Semantic UI components, such as exposing sensitive data or disabling security features.
5. Establish a regular schedule for monitoring vendor advisories and updating third-party libraries to address new vulnerabilities.

#47 Low Form Detection (severity: info) [tags: form, misc, miscellaneous, vuln]

Host: 172.19.0.3

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability identifies web forms on host 172.19.0.3, suggesting the presence of a web application or service with input fields. Real-world attack scenarios include injection attacks like SQL injection, cross-site scripting (XSS), or brute-force attacks on authentication forms. Business impact may involve unauthorized access to sensitive data, service disruption, or reputational damage due to potential breaches. Although severity is informational, it highlights areas for security review to prevent escalation into critical threats.

Exploitability rationale: The finding is informational, on an internal host with no specific vulnerabilities or public exploits, requiring high attack complexity to exploit potential issues.

Remediation: Secure detected web forms through validation, protection, and access management to mitigate potential risks.

1. Locate and inventory the detected form on the host 172.19.0.3.
2. Assess the form's purpose and the sensitivity of data it handles.
3. Implement server-side input validation to sanitize user inputs and prevent injection attacks.
4. Add Cross-Site Request Forgery (CSRF) tokens to the form for protection against CSRF attacks.
5. Ensure the form is served over HTTPS to encrypt data in transit.
6. Configure access controls to restrict form submission to authorized users if necessary.
7. Set up logging and monitoring for form submissions to detect anomalies.
8. Conduct regular security reviews and penetration testing on the form.

Ranked Summary

Rank	Score	Exploitability	Finding	Host
1	0.66	Medium	Debug Endpoint pprof - Exposure Detection (severity: high) [tags: cve2019, cve, debug, kubernetes, kubelet, devops, unauth, disclosure, vkev, vuln]	172.19.0.3
2	0.65	High	DVWA Default Login (severity: critical) [tags: dvwa, default-login, vuln]	172.19.0.2
3	0.63	Medium	Grafana - XSS / Open Redirect / SSRF via Client Path Traversal (severity: high) [tags: cve, cve2025, grafana, redirect, unauth, oss, vkev, vuln]	172.19.0.3
4	0.555	Medium	Nova noVNC - Open Redirect (severity: medium) [tags: cve2021, cve, redirect, novnc, seclists, openstack, vuln]	172.19.0.3
5	0.515	Medium		172.19.0.2

Rank	Score	Exploitability	Finding	Host
			Sensitive Configuration Files Listing - Detect (severity: medium) [tags: config, listing, exposure, edb, vuln]	
6	0.435	Low	Open Redirect - Detection (severity: medium) [tags: redirect, generic, vuln]	172.19.0.3
7	0.38	Low	Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.2
8	0.38	Low	Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.2
9	0.38	Low	WAF Detection (severity: info) [tags: waf, tech, misc, discovery]	172.19.0.2
10	0.38	Low	Gitignore Config - Detect (severity: info) [tags: exposure, tenable, config, git, vuln]	172.19.0.2
11	0.38	Low	FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]	172.19.0.2
12	0.38	Low	Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]	172.19.0.2
13	0.38	Low	Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]	172.19.0.2
14	0.38	Low	README.md file disclosure (severity: info) [tags: exposure, markdown, files, vuln]	172.19.0.2
15	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
16	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
17	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
18	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
19	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
20	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
21	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
22	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
23	0.38	Low		172.19.0.2

Rank	Score	Exploitability	Finding	Host
			HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	
24	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
25	0.38	Low	robots.txt endpoint prober (severity: info) [tags: miscellaneous, misc, generic, discovery]	172.19.0.2
26	0.38	Low	robots.txt file (severity: info) [tags: miscellaneous, misc, generic, vuln]	172.19.0.2
27	0.38	Low	Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.3
28	0.38	Low	Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.3
29	0.38	Low	SNMPv3 Fingerprint - Detect (severity: info) [tags: js, udp, network, snmp]	172.19.0.3
30	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
31	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
32	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
33	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
34	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
35	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
36	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
37	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
38	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.3
39	0.38	Low	FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]	172.19.0.3
40	0.38	Low	Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]	172.19.0.3
41	0.38	Low	Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]	172.19.0.3

Rank	Score	Exploitability	Finding	Host
42	0.38	Low	Wordpress Plugin Detection (severity: info) [tags: tech, wp, wordpress, plugin, detection, wp-plugin, discovery]	172.19.0.3
43	0.38	Low	Wordpress Plugin Detection (severity: info) [tags: tech, wp, wordpress, plugin, detection, wp-plugin, discovery]	172.19.0.3
44	0.38	Low	Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]	172.19.0.3
45	0.38	Low	FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]	172.19.0.3
46	0.38	Low	Semantic UI Framework - Detect (severity: info) [tags: tech, semantic-ui, discovery]	172.19.0.3
47	0.38	Low	Form Detection (severity: info) [tags: form, misc, miscellaneous, vuln]	172.19.0.3

Risk score = (CVSS/10 × 0.5) + (Exploitability × 0.3) + (Asset × 0.2). Exploitability: High=1.0, Medium=0.5, Low=0.1.